SYSTEMS PERFORMANCE

EVALUATING LOCK WAIT POLICIES

A SYSTEMS PERFORMANCE EVALUATION OF SPIN, MUTEX AND ADAPTIVE WAITING

CONTENTS

01 | LOCK CONTENTION

02 | METHODOLOGY

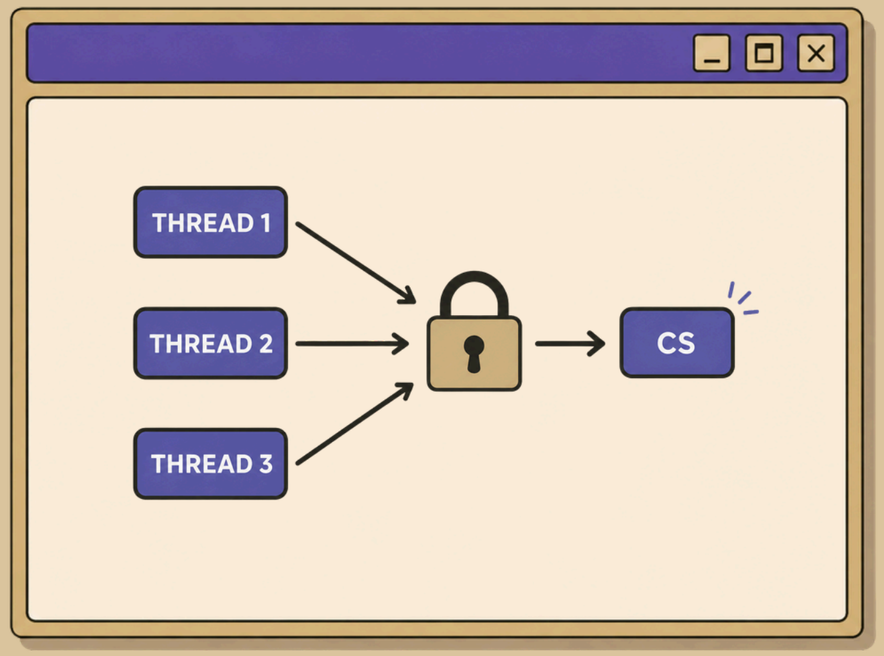
03 | EXPERIMENTAL PROCEDURE

04 | RESULTS & ANALYSIS

05 | CONCLUSION

LOCK CONTENTION

HOW CONTENTION ARISES



01 - LOCK CONTENTION

PARALLEL EXECUTION

Multiple threads can execute at the same time to perform work concurrently.

CRITICAL SECTION

Some operations access shared data and cannot safely execute at the same time.

MUTUAL EXCLUSION

A lock allows only one thread at a time to enter this critical section.

LOCK CONTENTION

If another thread requests the lock while it is held, contention occurs and the thread must wait.

LOCK POLICIES

1. SPINLOCK
2. MUTEX
3. ADAPTIVE

01 - LOCK CONTENTION

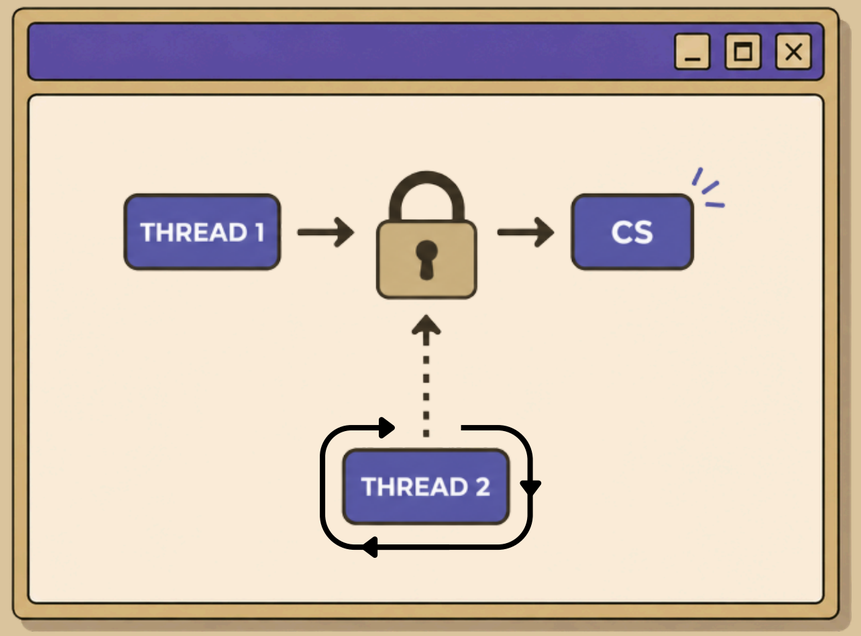
> Lock policies determine how threads wait to acquire a lock.

> Different policies balance CPU usage against lock-acquisition latency.

> Common approaches include spinning, sleeping, or an adaptive combination of both.

LOCK POLICIES

01 - LOCK CONTENTION



> Stays on the CPU

> Repeatedly checks the lock

> Acquires the lock as soon as it is free

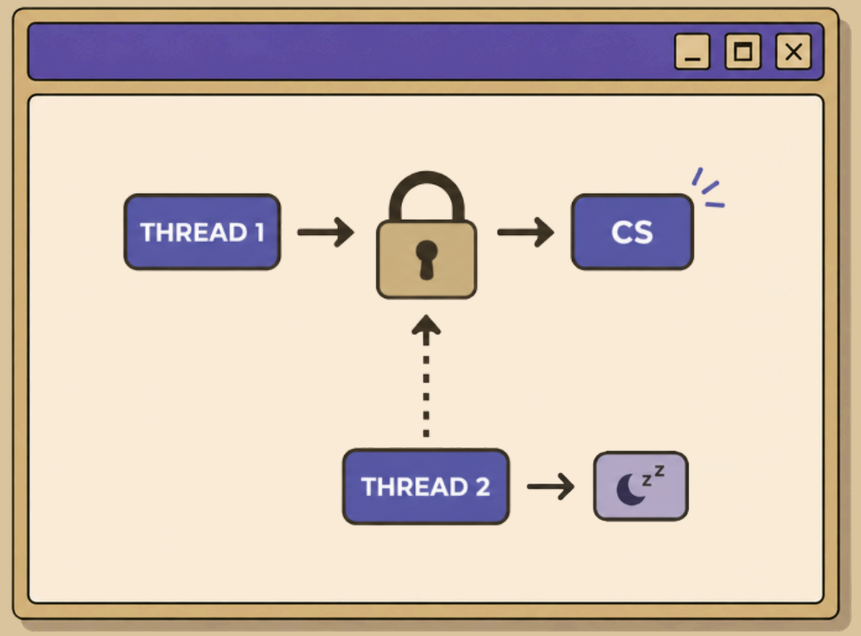
1. SPINLOCK

2. MUTEX

3. ADAPTIVE

LOCK POLICIES

01 - LOCK CONTENTION



> Blocks when the lock is busy

> Gives up the CPU

> Wakes and retries later

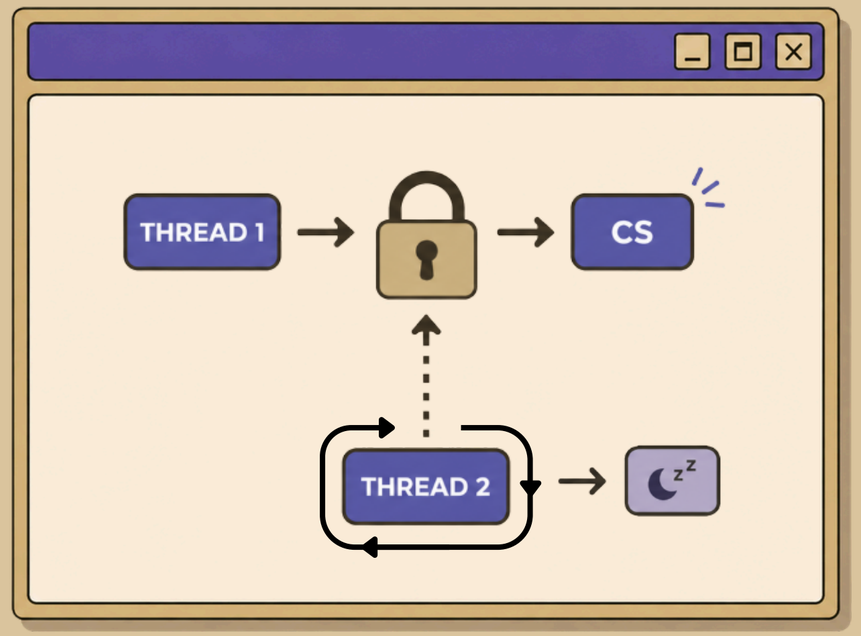
1. SPINLOCK

2. MUTEX

3. ADAPTIVE

LOCK POLICIES

01 - LOCK CONTENTION



> Spins for a short time

> Blocks if the lock stays busy

> Wakes and retries later

1. SPINLOCK

2. MUTEX

3. ADAPTIVE

THE PERFORMANCE TRADE-OFF

01 - LOCK CONTENTION

SPINLOCK

ADAPTIVE

MUTEX

STAYS ON CPU

> Avoids wakeup overhead

> Consumes CPU while waiting

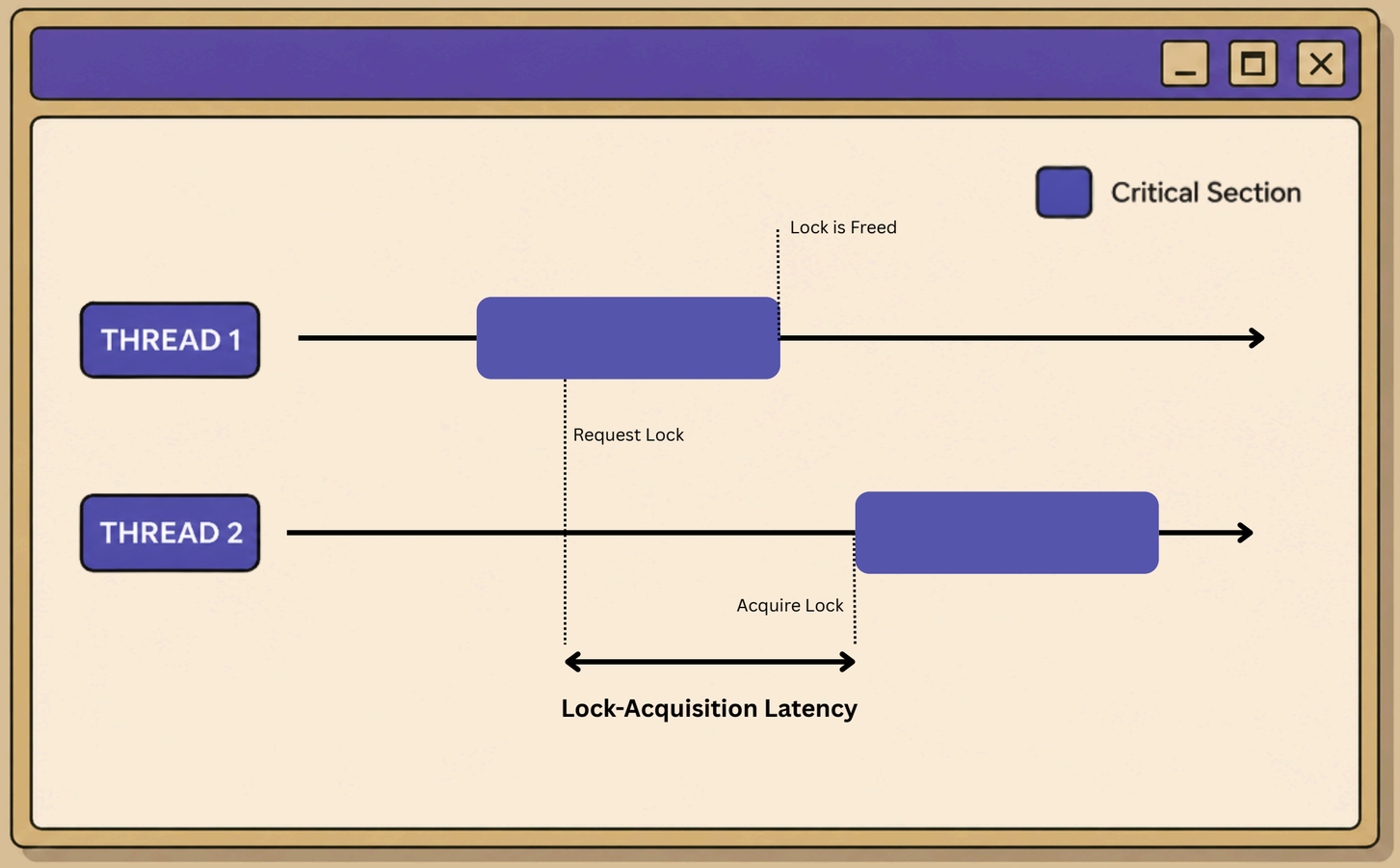
GIVES UP CPU

> Pays wakeup overhead

> Frees CPU while blocked

HOW POLICIES ARE COMPARED

01 - LOCK CONTENTION



METHODOLOGY

METHODOLOGICAL FRAMEWORK

02 - METHODOLOGY

Scientific Method

Question → **Model** → **Hypothesis** → **Prediction** → **Test** → **Analysis** → **Conclusion**

- **Question**: How does lock wait policy affect lock-acquisition latency as thread contention increases?
- **Model**: Identify the mechanisms that should determine how each wait policy behaves under contention.
- **Hypothesis**: Turn the expected lock behaviours into falsifiable claims about latency under contention.
- **Prediction**: Define the observable patterns expected in p50, p99 and distributions as contention increases.
- **Test**: Vary wait policy and contention while holding other influential conditions constant.
- **Analysis**: Compare latency distributions across policies, contention levels and repeats against the predictions.
- **Conclusion**: Decide which hypotheses the evidence supports, rejects or only partially supports, within the tested system.

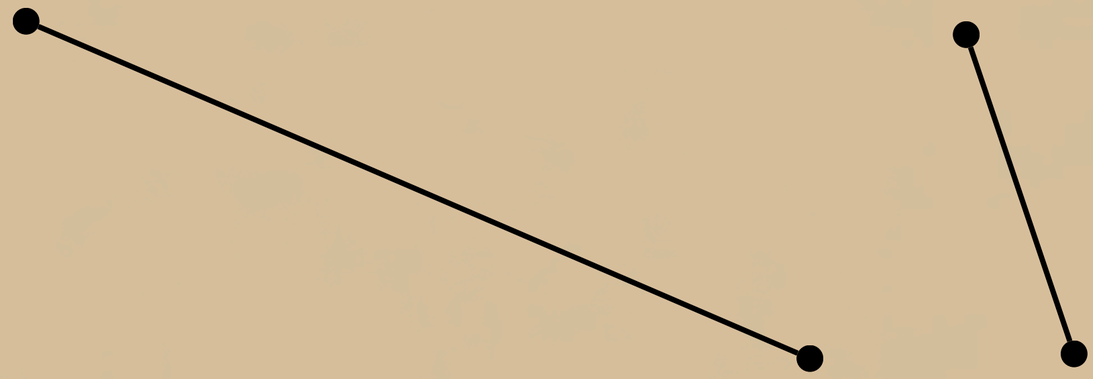
PERFORMANCE-SPECIFIC METHODS

Microbenchmarking

Isolate lock behaviour
Use a controlled synthetic workload
Minimise unrelated system effects

Latency Analysis

Measure latency in time
Analyse p50, p99 and distributions
Avoid hiding behaviour in averages



Question → Model → Hypothesis → Prediction → **Test** → **Analysis** → Conclusion

DEFINING THE PROBLEM

How does the lock wait policy affect lock-acquisition latency under increasing thread contention?

What changes?

Wait policy

Thread Contention

What is measured?

Lock-acquisition latency

What stays fixed?

Critical-section length

Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

PERFORMANCE MODEL

Spinlock

Known Behaviour
> Stays runnable and is repeatedly checking the lock

Performance Mechanism
> Avoids blocking and wake-up overhead
> Consumes CPU while waiting

Expected Results
> Will benefit in short waits when the CPU is readily available

Adaptive

Known Behaviour
> Spins initially, then blocks after a certain amount of time/rotations

Performance Mechanism
> Can avoid blocking for short waits
> But sleeps for longer waits

Expected Results
> Behaviour will change as contention increases

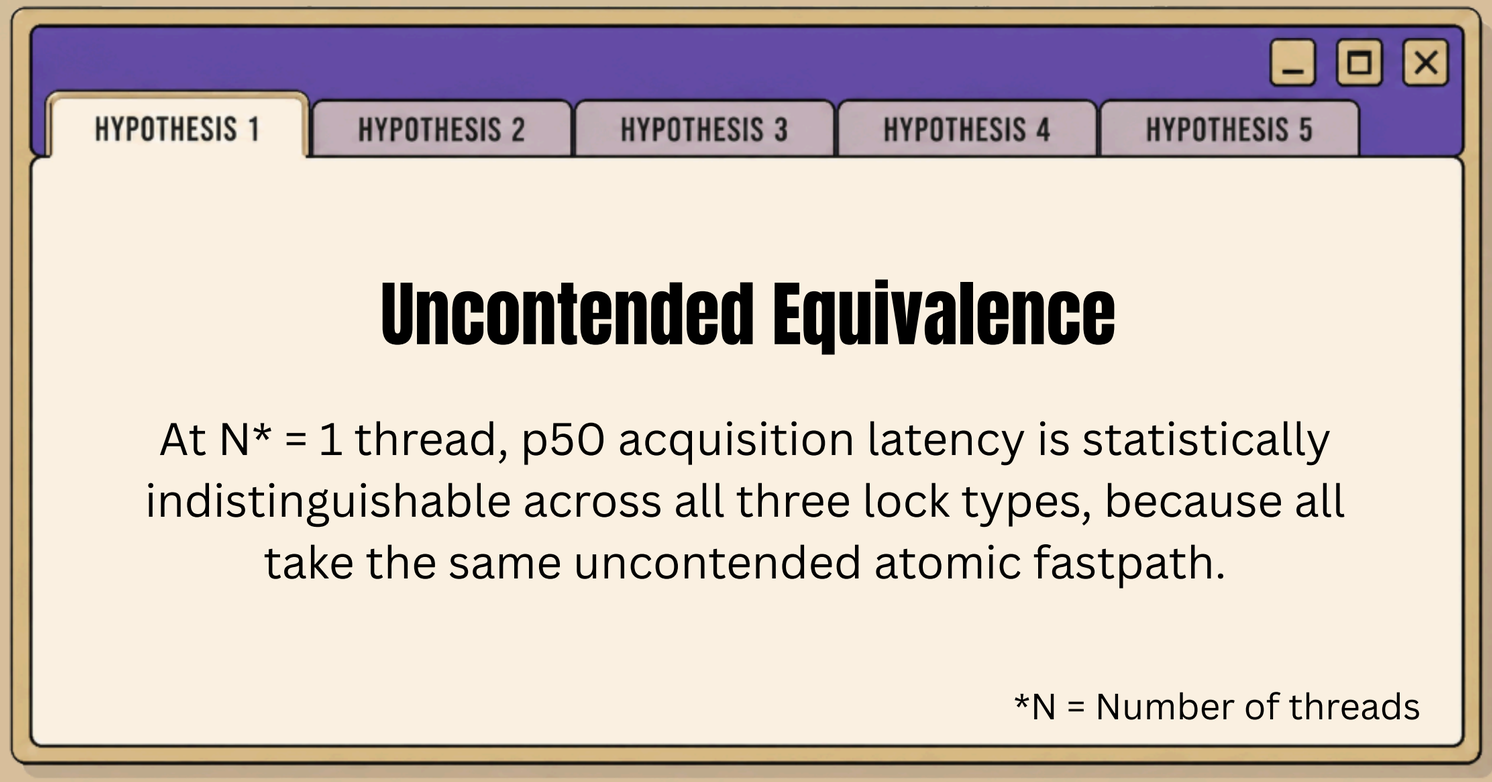
Mutex

Known Behaviour
> Blocks and gives up the CPU when contended

Performance Mechanism
> Frees CPU while waiting
> Acquisition can involve wake-up and scheduling

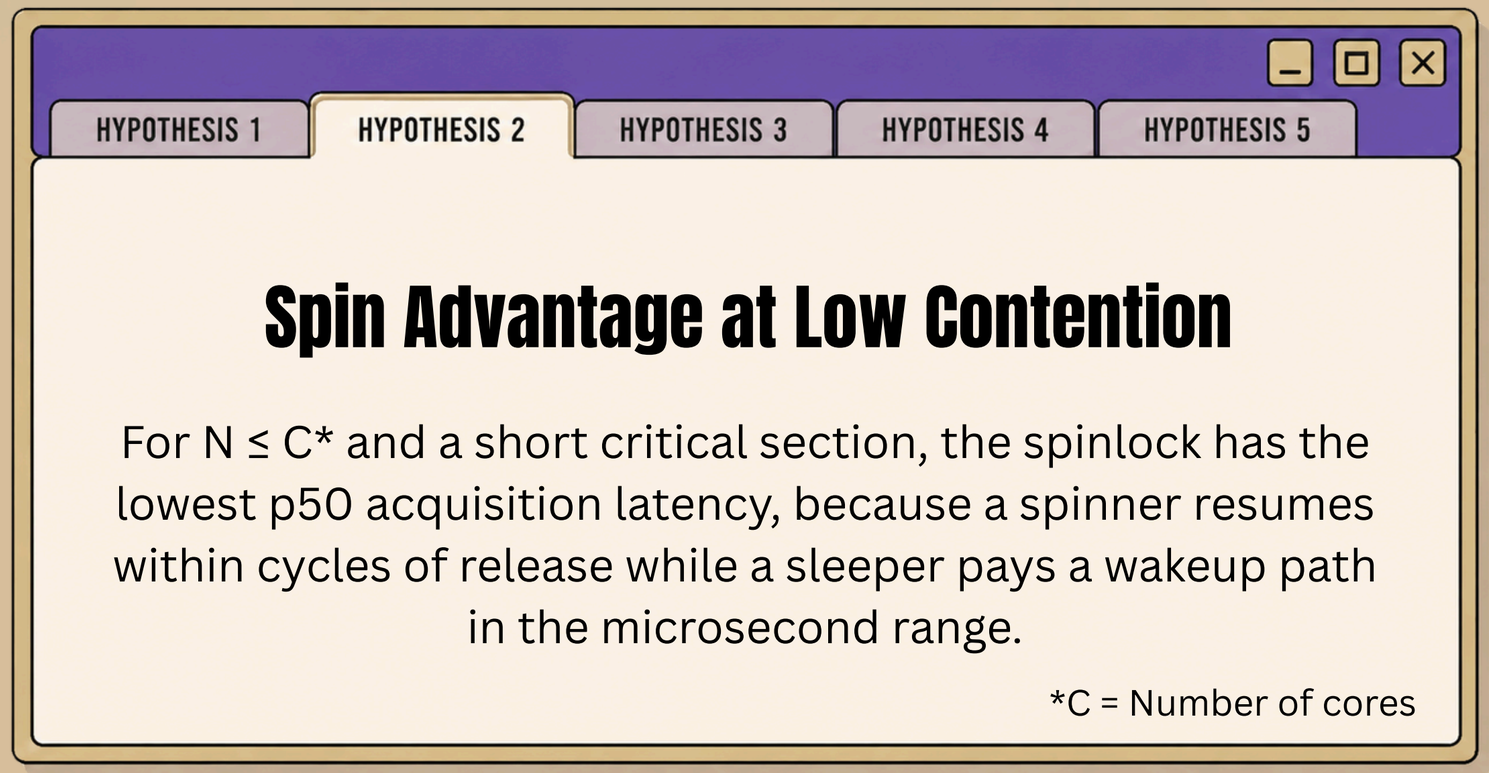
Expected Results
> Will incur a higher latency when the blocking path is taken

BUILDING THE HYPOTHESES



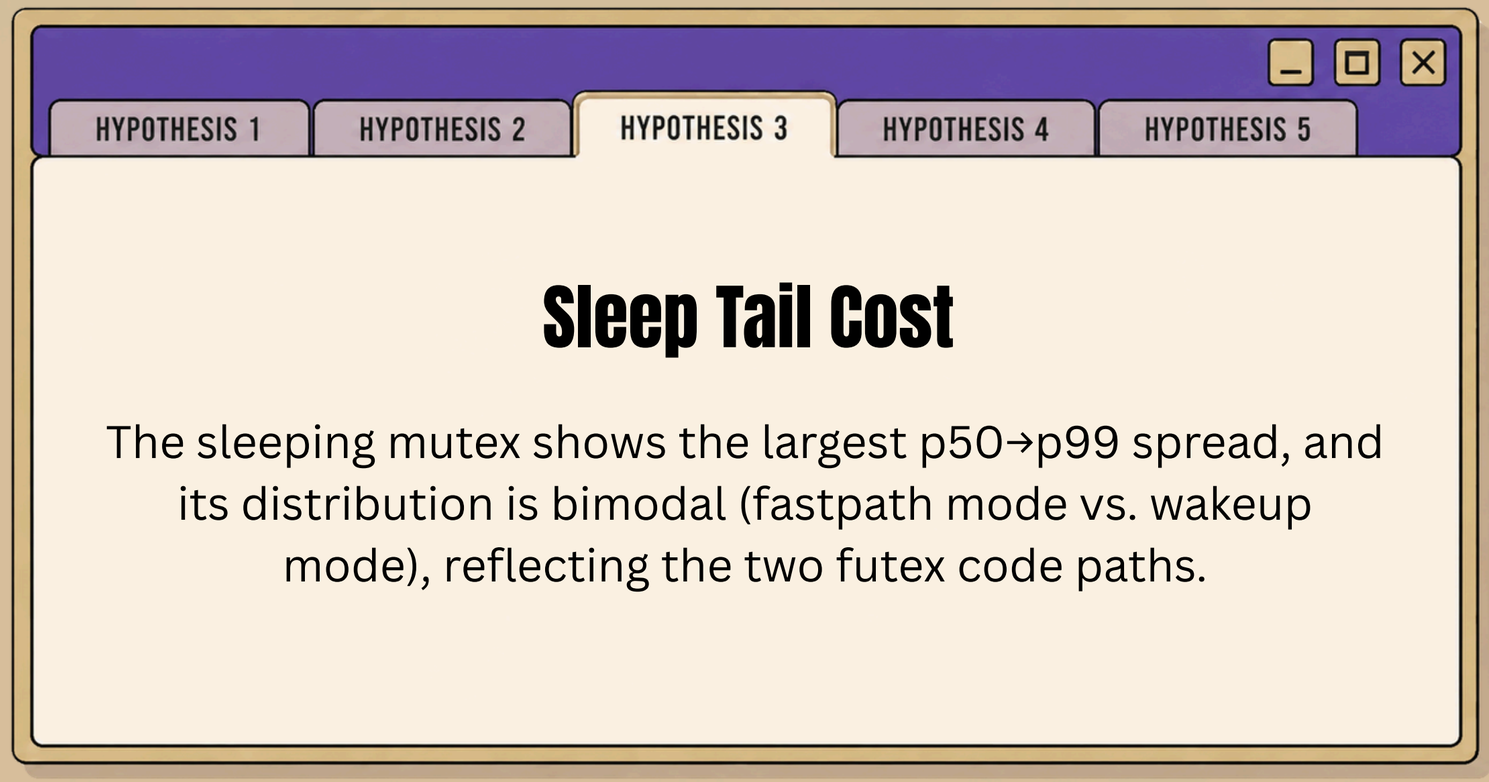
Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

BUILDING THE HYPOTHESES



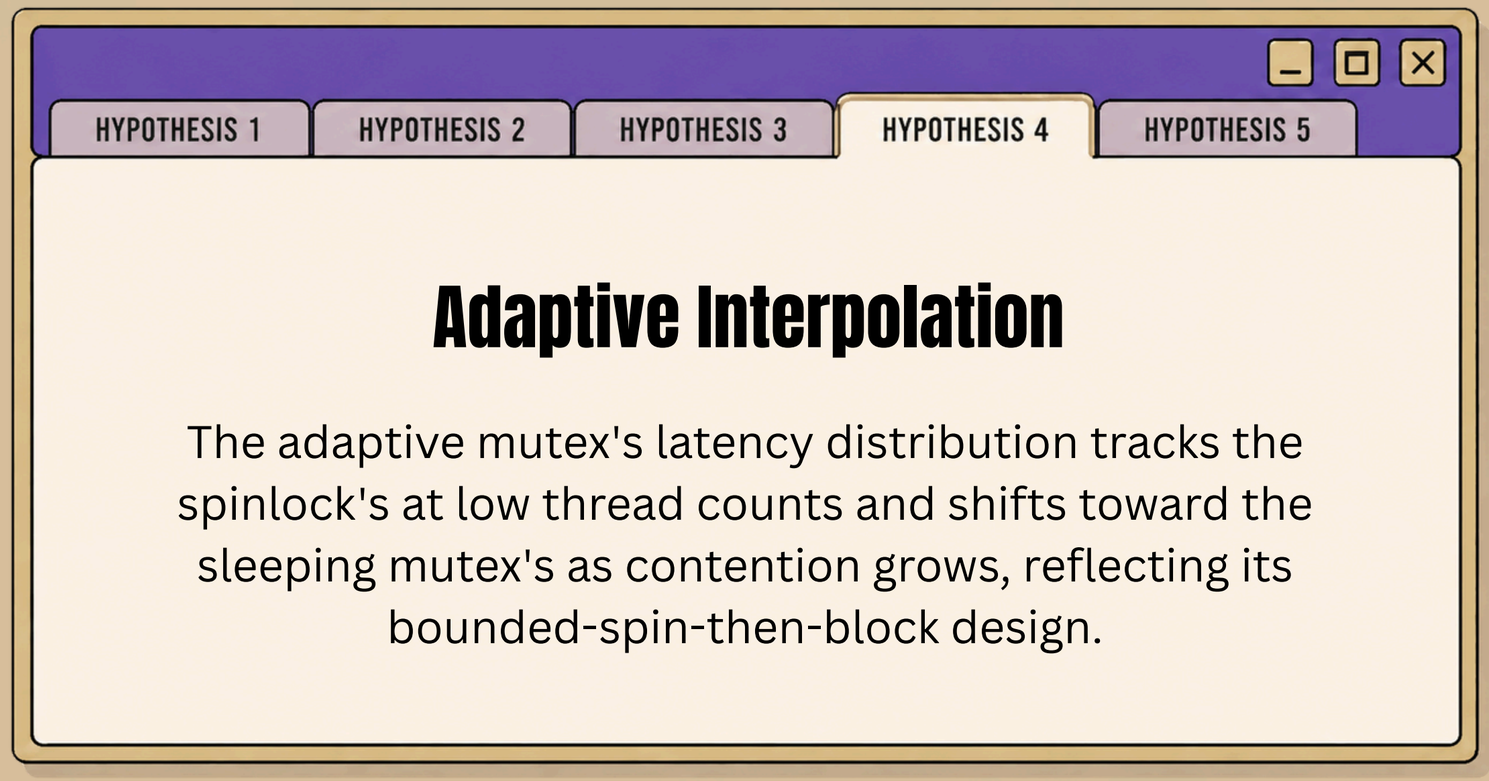
Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

BUILDING THE HYPOTHESES



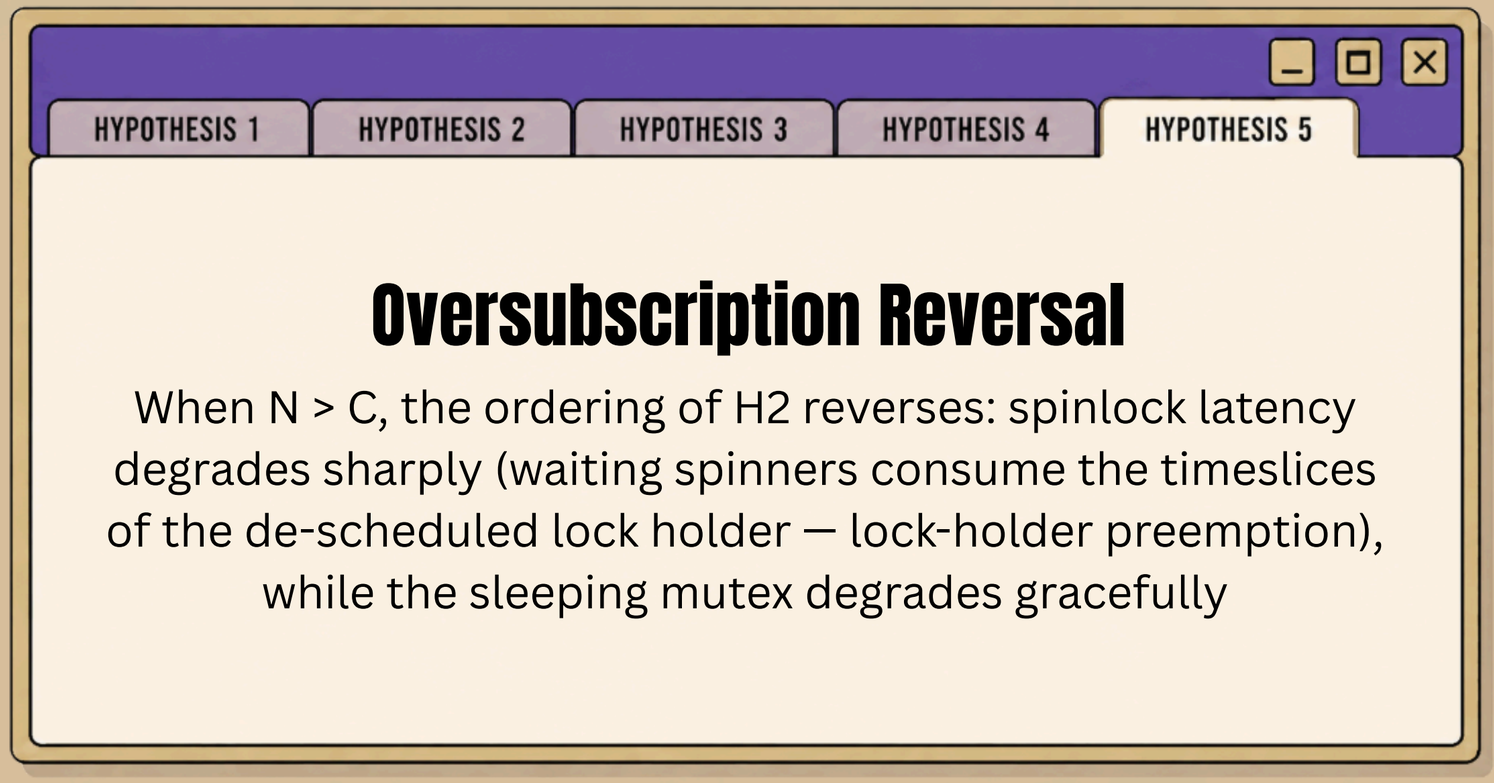
Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

BUILDING THE HYPOTHESES



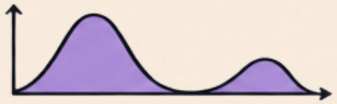
Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

BUILDING THE HYPOTHESES



Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

WHAT WE EXPECT TO OBSERVE

H1	Uncontended Equivalence		Similar latency across all three policies at N = 1
H2	Spin Advantage at Low Contention		Spin lowest, adaptive intermediate, mutex highest at N ≤ C
H3	Sleep Tail Cost		Mutex shows distinct fast and slow lock-acquisition latencies
H4	Adaptive Interpolation		Adaptive shifts from spin-like toward mutex-like behaviour as thread count increases
H5	Oversubscription Reversal		Spin becomes highest latency at N > C, reversing the earlier ordering

Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

EXPERIMENTAL PROCEDURE

ISOLATING THE PHENOMENON

Macro-Benchmark

Real Application Workload

Lock behaviour interacts with:

- Application logic
- Memory behaviour
- I/O
- scheduling
- caches
- ...

Realistic, but many factors influence the result

Micro-Benchmark

Artificial Controlled Workload

Threads → One shared lock → Fixed CS → Repeat

Less realistic, but easier to isolate and attribute

Test: Design → Setup → Execution → Validation

THE BENCHMARK

03 - EXPERIMENTAL PROCEDURE

Target Operation

Lock Acquisition

Single Thread
Requests shared lock
Waits until Acquisition

Test Dimensions

Wait Policy:

Spin / Mutex / Adaptive

Thread Contention

Increasing worker count

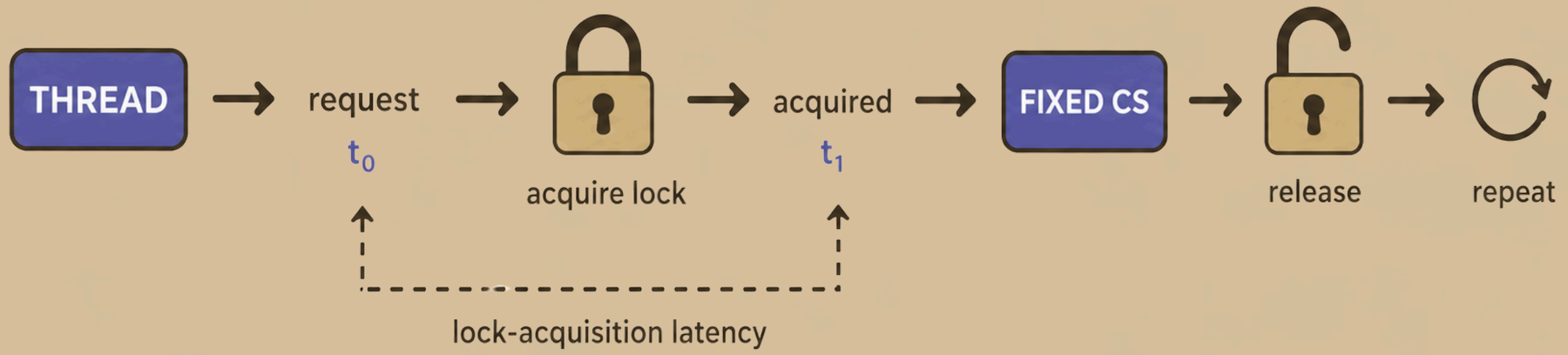
Response

Lock Acquisition Latency

Request → Acquire

Test: | Design → Setup → Execution → Validation

ONE ITERATION OF THE MICRO-BENCHMARK



Test: | Design → Setup → Execution → Validation

RETAINING EVERY ACQUISITION

Traditional Micro-benchmarking

Aggregate Results

$$\text{Average Latency} = \frac{\text{Total Runtime}}{\text{Operation Count}}$$

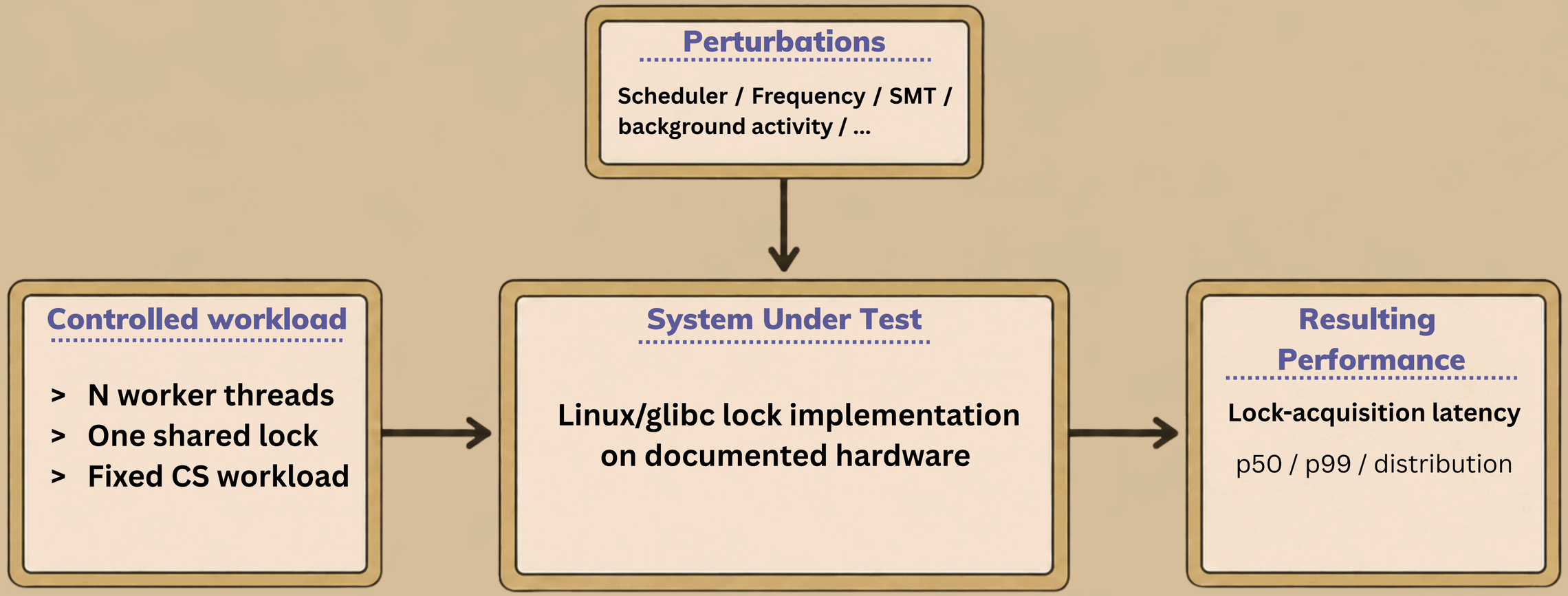
Micro-Benchmark with Latency Analysis

Keep Every Acquisition

p50	Typical Behaviour
p99	Tail Behaviour
Distribution	Overall shape

Test: Design → Setup → Execution → Validation

SYSTEM UNDER TEST



Test: Design → Setup → Execution → Validation

SYSTEM UNDER TEST

SUT

Linux / glibc locks

Spin
pthread_spinlock_t

Mutex
pthread_mutex_t

Adaptive
PTHREAD_MUTEX_ADAPTIVE_NP

Environment

Hardware

Surface Laptop 4
i5-1135G7
4 physical cores
1 socket
SMT inactive
Bare metal

Software

Ubuntu 26.04.
Kernel 7.0.0-30
glibc 2.43
GCC 15.2

CPU 0
Control

CPU 1
Worker

CPU 2
Worker

CPU 3
Worker

Test: | Design → Setup → Execution → Validation

CONTROLLING THE PHENOMENON - I

03 - EXPERIMENTAL PROCEDURE

Confounder	Effect if uncontrolled	Mitigation	Validation
Critical Section Length	Changes the spin-sleep trade-off	200ns target Selected by the pilot and frozen	Realised CS within ±20% every run
Frequency scaling / boost	Frequency changes distort nanosecond-scale timing	Performance governor Boost inactive	CPU state recorded in preflight
SMT	Sibling thread can compete with the lock holder	SMT inactive One thread per physical core	SMT/topology state recorded

Test: Design → Setup → Execution → Validation

CONTROLLING THE PHENOMENON - II

Confounder	Effect if uncontrolled	Mitigation	Validation
Core Topology / CPU Placement	Migration/cache distance can change acquisition cost	Workers pinned to CPUs 1–3 on one socket	CPU placement checked per run
Virtualisation / Steal	Hypervisor preemption adds unrelated delay	Bare-metal Linux only	No virtualisation detected in preflight
False Sharing of Benchmark data	Shared cache lines create extra contention	Per-thread data 64-byte aligned and padded	Padding present in final benchmark

Test: Design → Setup → Execution → Validation

CONTROLLING THE MEASUREMENT

03 - EXPERIMENTAL PROCEDURE

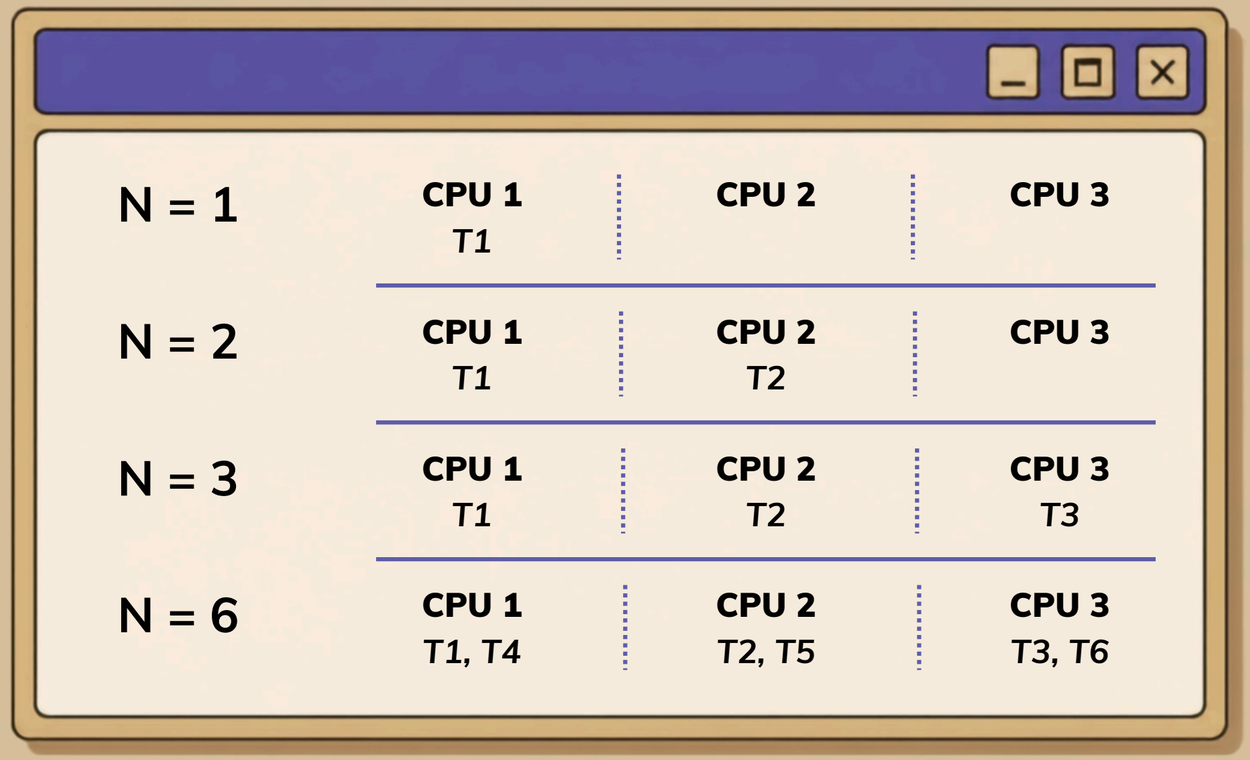
Confounder	Effect if uncontrolled	Mitigation	Validation
Observer Effect of Tracing	Hot-path tracing can exceed the latency being measured	In-process timestamps No hot-path tracing	Same measurement path for every arm
Timer Overhead and Resolution	Timestamp cost can distort short acquisitions	CLOCK_MONOTONIC characterised separately	Overhead ≈ 23 ns Rule passed every run
Compiler Optimisation	Compiler may remove/reorder synthetic work	volatile loop + memory barrier	Exact source/binary preserved

Test: Design → Setup → Execution → Validation

CONTROLLING THE STATISTICS

Confounder	Effect if uncontrolled	Mitigation	Validation
Cold-start transients	Startup effects distort early samples	10,000 warm-up acquisitions per worker	Measurement begins after warm-up
Tail noise / run-to-run variation	Rare interruptions can dominate p99	1,000,000 samples/config + 5 repeats	Sample counts and context switches recorded
Implementation specificity	Results depend on the software and hardware implementation	Same SUT, kernel, glibc and compiler	Versions and artefacts recorded

CPU / THREAD PLACEMENT



	CPU 1	CPU 2	CPU 3
N = 1	*T1*		
N = 2	*T1*	*T2*	
N = 3	*T1*	*T2*	*T3*
N = 6	*T1, T4*	*T2, T5*	*T3, T6*

Test: | Design → Setup → Execution → Validation

CONTROLLING THE EXPERIMENTAL ENVIRONMENT

configure_experiment.sh

```
# Select one allowed logical CPU from each physical core on that socket
CPU_LIST="$(
    lscpu -p=CPU,CORE,SOCKET,ONLINE |
    awk -F',' -v socket="$SOCKET" -v allowed="$ALLOWED_CPUS" '
        BEGIN {
            n = split(allowed, cpus, ",")
            for (i = 1; i <= n; i++)
                permitted[cpus[i]] = 1
        }

        !/^#/ && toupper($4) == "Y" && $3 == socket && permitted[$1] {
            key = $2 ":" $3

            if (!seen[key]++)
                print $1
        }
    ' |
    paste -sd, -
)"

if [ -z "$CPU_LIST" ]; then
    echo "Could not select measurement CPUs."
    exit 1
fi

CORE_COUNT="$(echo "$CPU_LIST" | awk -F',' '{print NF}')"

echo "Process allowed CPUs: $ALLOWED"
echo "Detected socket: $SOCKET"
echo "Physical cores available on socket: $CORE_COUNT"
echo "One logical CPU per core: $CPU_LIST"
```

Topology-aware CPU placement

Select one logical CPU per physical core on a single socket and write it into experiment.conf

CODE WALKTHROUGH

```
# Reserve the first physical core for OS/background work if at least three cores exist
if [ "$CORE_COUNT" -ge 3 ]; then

    HOUSEKEEPING_CPU="${CPU_LIST%%,*}"
    CPU_LIST="${CPU_LIST#*,}"
    PHYSICAL_CORES=$((CORE_COUNT - 1))

    echo
    echo "Reserving CPU $HOUSEKEEPING_CPU for system/background activity."

else

    PHYSICAL_CORES="$CORE_COUNT"

    echo
    echo "Only $CORE_COUNT physical cores available - no core reserved for housekeeping."

fi
```

Reserve one core for OS/background activity by removing it from CPU_LIST

*"We're aware of **isolcpus/nohz_full**, but since we measured involuntary context switches **(ivcsw)** per run and they stayed negligible, hard isolation wasn't necessary."*

STANDARDISING EACH TEST

03 - EXPERIMENTAL PROCEDURE

1,000,000
Measured Acquisitions
per configuration

10,000
Warm-up Acquisitions
per worker

5
Independent Repeats
per configuration

200 ns
Critical-section target
pilot-selected and frozen

Test: Design → Setup → Execution → Validation

PREFLIGHT CHECKS

Check	Required State	Result
Environment	Native / bare-metal Linux	PASS
CPU configuration	Valid measurement CPU set	PASS
Core topology	Distinct physical cores, one socket	PASS
SMT	Disabled	PASS
CPU governor	performance	PASS
Turbo / boost	Inactive	PASS
Benchmark	Functional tests pass	PASS

Test: Design → Setup → Execution → Validation

PREFLIGHT CHECKS

CODE WALKTHROUGH

```
if [ -f /sys/devices/system/cpu/smt/active ]; then
    smt="$(cat /sys/devices/system/cpu/smt/active)"
    if [ "$smt" = "0" ]; then
        pass "SMT disabled"
    elif [ "$FINAL_MODE" -eq 1 ]; then
        fail "SMT is enabled"
    else
        warn "SMT is enabled - disable before the final experiment"
    fi

else
    if [ "$FINAL_MODE" -eq 1 ]; then
        fail "SMT state could not be determined"
    else
        warn "SMT state could not be determined"
    fi

fi
```

```
if [ "$governor_missing" -eq 1 ] || [ -z "$unique_governors" ]; then
    if [ "$FINAL_MODE" -eq 1 ]; then
        fail "CPU governor state unavailable on one or more measurement CPUs"
    else
        warn "CPU governor information unavailable"
    fi
elif [ "$unique_governors" = "performance" ]; then
    pass "Measurement CPU governor is performance"
else
    if [ "$FINAL_MODE" -eq 1 ]; then
        fail "Measurement CPU governor is not performance: $unique_governors"
    else
        warn "Measurement CPU governor is not performance: $unique_governors"
    fi
fi
```

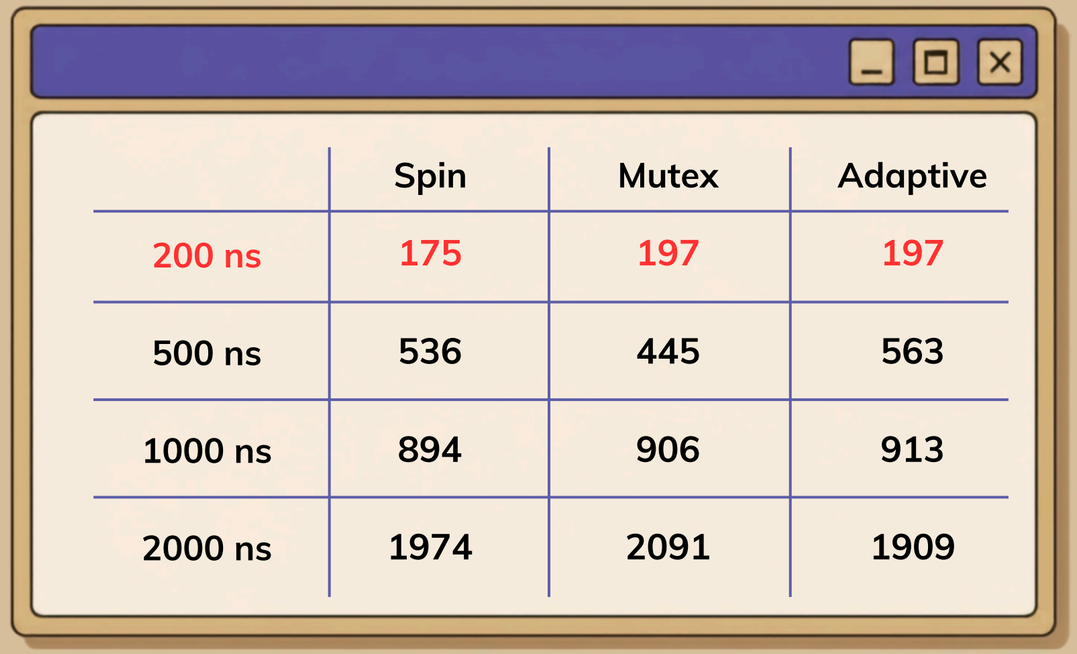
```
# Turbo / boost state

BOOST_STATE="unknown"
BOOST_SOURCE=""

if [ -f /sys/devices/system/cpu/intel_pstate/no_turbo ]; then
    BOOST_SOURCE="intel_pstate/no_turbo"
    if [ "$(cat /sys/devices/system/cpu/intel_pstate/no_turbo)" = "1" ]; then
        BOOST_STATE="disabled"
    else
        BOOST_STATE="enabled"
    fi

elif [ -f /sys/devices/system/cpu/cpufreq/boost ]; then
    BOOST_SOURCE="cpufreq/boost"
    if [ "$(cat /sys/devices/system/cpu/cpufreq/boost)" = "0" ]; then
        BOOST_STATE="disabled"
    else
        BOOST_STATE="enabled"
    fi
fi

if [ "$BOOST_STATE" = "disabled" ]; then
    pass "CPU turbo/boost disabled ($BOOST_SOURCE)"
elif [ "$BOOST_STATE" = "enabled" ]; then
    if [ "$FINAL_MODE" -eq 1 ]; then
        fail "CPU turbo/boost is enabled ($BOOST_SOURCE)"
    else
        warn "CPU turbo/boost is enabled - disable before final experiment"
    fi
else
    if [ "$FINAL_MODE" -eq 1 ]; then
        fail "CPU turbo/boost state could not be determined"
    else
        warn "CPU turbo/boost state could not be determined"
    fi
fi
```

Control CPU execution conditions

Require SMT to be disabled, use the performer governer, and disable turbo/boost before the final measurement

These are just a few examples of the control checks done before execution

SELECTING THE CS LENGTH



	Spin	Mutex	Adaptive
200 ns	175	197	197
500 ns	536	445	563
1000 ns	894	906	913
2000 ns	1974	2091	1909

03 - EXPERIMENTAL PROCEDURE

Candidates

200 ns | 500 ns | 1000 ns | 2000 ns

↓

Pilot Test

N = 2 | Spin/Mutex/Adaptive

↓

Acceptance Rule

Realised CS within ±20%

Target ≥ 5× timer overhead

↓

Selected

200 ns

↓

Final Preflight

Test: | Design → Setup → Execution → Validation

CONSTRUCTING THE MICROBENCHMARK

CODE WALKTHROUGH

lockbench.c

```c
static void report_timer(uint64_t *overhead_out) {
    struct timespec res;
    clock_getres(CLOCK_MONOTONIC, &res);

    enum { CAL = 1000000 };
    uint64_t t0 = now_ns();
    for (int i = 0; i < CAL; i++) {
        volatile uint64_t sink = now_ns();  // volatile: keep the call (C8)
        (void)sink;
    }
    uint64_t per_call = (now_ns() - t0) / CAL;
    *overhead_out = per_call;
    printf("# timer: CLOCK_MONOTONIC resolution=%ldns, clock_gettime overhead=%luns\n",
           res.tv_nsec, (unsigned long)per_call);
}
```

```c
// Calibrated critical section (C1, C8)

static uint64_t cs_spins_per_iter;  // busy-loop iterations for cfg.cs_ns
static uint64_t realised_cs_ns;

static inline void busy_loop(uint64_t n) {
    for (volatile uint64_t i = 0; i < n; i++)
        __asm__ __volatile__("" ::: "memory");   // barrier: not optimisable away
}

static void calibrate_cs(void) {
    // find loop iterations per ns, then scale to target
    enum { PROBE = 50000000ull };
    uint64_t t0 = now_ns();
    busy_loop(PROBE);
    uint64_t dt = now_ns() - t0;
    double iters_per_ns = (double)PROBE / (double)dt;
    cs_spins_per_iter = (uint64_t)(iters_per_ns * (double)cfg.cs_ns);
    if (cs_spins_per_iter == 0) cs_spins_per_iter = 1;
    enum { VERIFY = 10000, VERIFY_REPEATS = 9 };
    uint64_t verify_results[VERIFY_REPEATS];

    for (int r = 0; r < VERIFY_REPEATS; r++) {
        t0 = now_ns();

        for (int i = 0; i < VERIFY; i++) {
            busy_loop(cs_spins_per_iter);
        }

        verify_results[r] = (now_ns() - t0) / VERIFY;
    }

    // Sort the small result set and use the middle value.
    for (int i = 1; i < VERIFY_REPEATS; i++) {
        uint64_t value = verify_results[i];
        int j = i - 1;

        while (j >= 0 && verify_results[j] > value) {
            verify_results[j + 1] = verify_results[j];
            j--;
        }

        verify_results[j + 1] = value;
    }

    realised_cs_ns = verify_results[VERIFY_REPEATS / 2];

    printf("# cs: target=%ldns loop_iters=%lu realised=%luns\n",
           cfg.cs_ns,
           (unsigned long)cs_spins_per_iter,
           (unsigned long)realised_cs_ns);
}
```

Characterise and control the microbenchmark

Measure CLOCK_MONOTONIC resolution and timestamp overhead before collecting the results. This is a check to ensure that the measurement cost is small relative to the overall operation being timed.
Then calibrate a fixed critical section workload, and prevent the compiler optimiser from removing or reordering it.

FINAL EXPERIMENT EXECUTION

60 Runs

3 policies x 4 contention levels x 5 repeats

12 Unique Configurations

N = {1, 2, 3, 6} | Spin / Mutex / Adaptive

Individual Flow

Pin workers
To assigned CPUs

Warm-up
10,000 acquisitions / worker

Synchronise
Barrier to align workers

Measure
1,000,000 acquisitions

Save
Run Data

ONE MICROBENCHMARK ITERATION

CODE WALKTHROUGH

lockbench.c

Worker measurement process

Each worker is first pinned to its assigned CPU, then this placement is verified. Workers synchronise using a barrier, and then perform unmeasured warm-up acquisitions before being synchronised again before the actual measurements begin.

During the measured phase, timestamps are taking immediately before and immediately after lock_acquire(). This gives a ***proximate measurement*** *of the lock-acquisition latency. After the lock has been acquired the fixed critical-section workload runs, after which the lock is released and the latency sample is stored. This process is then repeated for all iterations.*

For validation we also recorded and measured the context switches to identify and scheduler interference.

```c
static void *worker_fn(void *arg) {
    struct worker *w = (struct worker *)arg;

    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(w->cpu, &set);
    int rc = pthread_setaffinity_np(pthread_self(), sizeof set, &set);
    if (rc != 0) {
        fprintf(stderr, "fatal: pinning thread %d to CPU %d failed: %s\n", w->id, w->cpu, strerror(rc));
        exit(1);
    }
    int actual_cpu = sched_getcpu();
    if (actual_cpu != w->cpu) {
        fprintf(stderr, "fatal: thread %d expected CPU %d but is running on CPU %d\n",
                w->id, w->cpu, actual_cpu);
        exit(1);
    }

    pthread_barrier_wait(&start_barrier);

    for (long i = 0; i < cfg.warmup; i++) {
        lock_acquire();
        busy_loop(cs_spins_per_iter);
        lock_release();
    }

    pthread_barrier_wait(&measure_barrier);

    struct rusage before, after;
    if (getrusage(RUSAGE_THREAD, &before) != 0) { perror("getrusage before"); exit(1); }
    w->measure_start_ns = now_ns();

    for (long i = 0; i < w->nsamples; i++) {
        uint64_t t0 = now_ns();
        lock_acquire();
        uint64_t t1 = now_ns();
        busy_loop(cs_spins_per_iter);
        lock_release();
        w->samples[i] = t1 - t0;
        w->done++;
    }

    w->measure_end_ns = now_ns();
    if (getrusage(RUSAGE_THREAD, &after) != 0) { perror("getrusage after"); exit(1); }
    w->vcsw = after.ru_nvcsw - before.ru_nvcsw;
    w->ivcsw = after.ru_nivcsw - before.ru_nivcsw;
    return NULL;
}
```

FROM MICROBENCHMARK TO FULL EXPERIMENT

CODE WALKTHROUGH

run_pilot.sh

```
# Select the shortest CS where all three runs calibrate within 20% of target
# and the target CS is at least 5x larger than the timer-call overhead.

SELECTED_CS=""

for cs in $CS_CANDIDATES; do

    valid=1
    max_timer=0

    while IFS=',' read -r target lock realised timer p50 p99; do

        [ "$target" = "$cs" ] || continue

        lower=$((cs * 80 / 100))
        upper=$((cs * 120 / 100))

        if [ "$realised" -lt "$lower" ] || [ "$realised" -gt "$upper" ]; then
            valid=0
        fi

        if [ "$timer" -gt "$max_timer" ]; then
            max_timer="$timer"
        fi

    done < <(tail -n +2 "$SUMMARY")

    if [ "$cs" -lt $((max_timer * 5)) ]; then
        valid=0
    fi

    if [ "$valid" -eq 1 ]; then
        SELECTED_CS="$cs"
        break
    fi
done
```

Pilot and then freeze the workload

Select the shortest reliable critical section using the calibration and timer-overhead criteria above.

run_experiments.sh

```
for ((repeat=1; repeat<=REPEATS; repeat++)); do

    # Rotate lock order between repeats to reduce ordering bias
    offset=$(((repeat - 1) % ${#LOCK_ARRAY[@]}))
    LOCK_ORDER=()

    for ((i=0; i<${#LOCK_ARRAY[@]}; i++)); do
        index=$(((i + offset) % ${#LOCK_ARRAY[@]}))
        LOCK_ORDER+=("${LOCK_ARRAY[$index]}")
    done

    # Odd repeats use ascending thread counts.
    # Even repeats use descending thread counts.

    if (( repeat % 2 == 1 )); then
        N_ORDER=("${THREAD_COUNTS[@]}")
    else
        N_ORDER=()
        for ((i=${#THREAD_COUNTS[@]} - 1; i>=0; i--)); do
            N_ORDER+=("${THREAD_COUNTS[$i]}")
        done
    fi

    printf 'Repeat %02d: N =' "$repeat" ' %s' "${N_ORDER[@]}"
    printf ' | locks = %s' "${LOCK_ORDER[@]}" '\n'
```

Systematic repeated testing

Run every lock and contention configuration across 5 repeats while rotating execution order. This reduces any time/order bias that may be present.

run_experiments.sh

```
# Run the benchmark
if "$BENCH_PATH" \
    --lock "$lock" \
    --threads "$n" \
    --cpus "$CPU_LIST" \
    --cs-ns "$RUN_CS_NS" \
    --samples "$RUN_SAMPLES" \
    --warmup "$RUN_WARMUP" \
    --csv "$samples_csv" \
    > "$stdout_txt" 2>&1; then

    echo "benchmark_exit=0" >> "$metadata_txt"
else
    rc=$?
    echo "benchmark_exit=$rc" >> "$metadata_txt"
    echo "FAIL benchmark exited with status $rc" > "$validation_txt"
    return 1
fi

# Validate the generated output
: > "$validation_txt"

# Extract ru-level calibration values for validity checking
realised="$(
    grep '^# cs:' "$stdout_txt" |
    sed -n 's/.*realised=\([0-9]*\)ns.*/\1/p'
)"

timer="$(
    grep '^# timer:' "$stdout_txt" |
    sed -n 's/.*overhead=\([0-9]*\)ns.*/\1/p'
)"
```

Execute with frozen controls and then validate

Run each configuration with the same CPU set, CS workload, sample count and warm-up. Then verify the calibration (shown above), sample integrity, CPU placement and scheduler activity.

VALIDATING THE FINAL EXPERIMENT

03 - EXPERIMENTAL PROCEDURE

60 / 60

VALID RUNS

12 configs x 5 reps

Sample Integrity

1,000,000
acquisitions / run

Critical section

within ±20%
rule satisfied

Measurement

≈23 ns
timer overhead

Execution

CPU Pinning
Verified

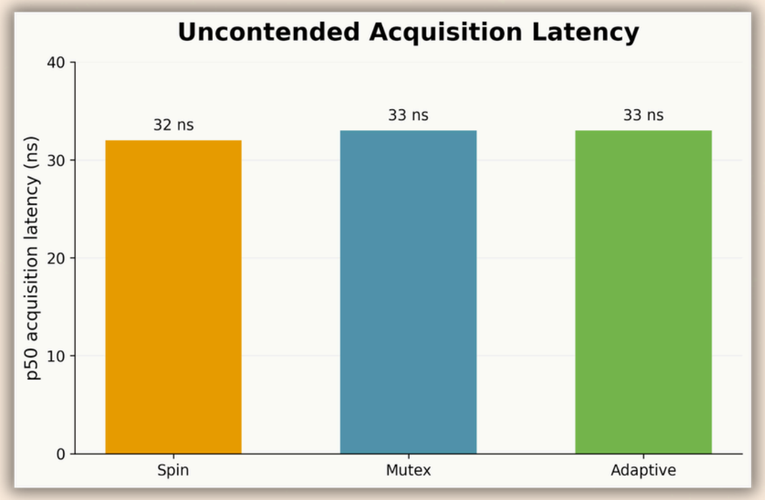
All planned configurations completed under the frozen experimental controls

Test: Design → Setup → Execution → Validation

RESULTS & ANALYSIS

H1: UNCONTENDED EQUIVALENCE

Prediction: Similar p50 latency at N = 1

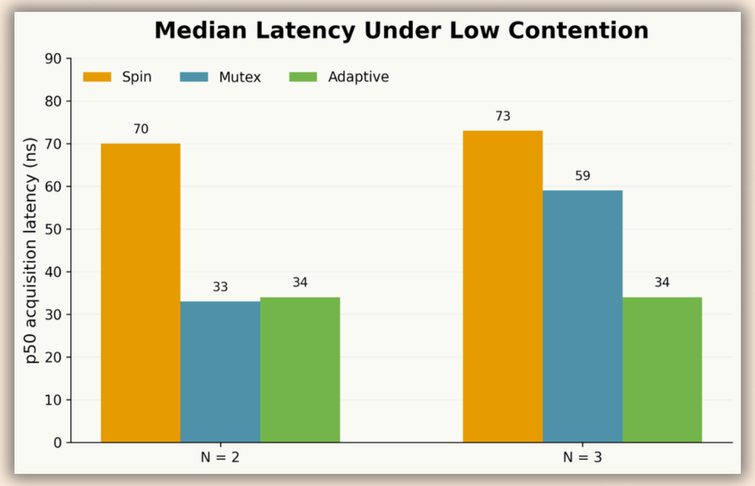


32-33 ns

across all three policies

H2: SPIN ADVANTAGE AT LOW CONTENTION

Prediction: Spinlock has the lowest p50 for short waits when N ≤ cores



~2.1× higher

spin p50 at N = 2

Spin ≈ 70 ns
Mutex / Adaptive ≈ 33-34 ns

INCUMBENT REACQUISITION

H2 Diagnosis

Why does spin behave differently even when N≤C?

≈99%

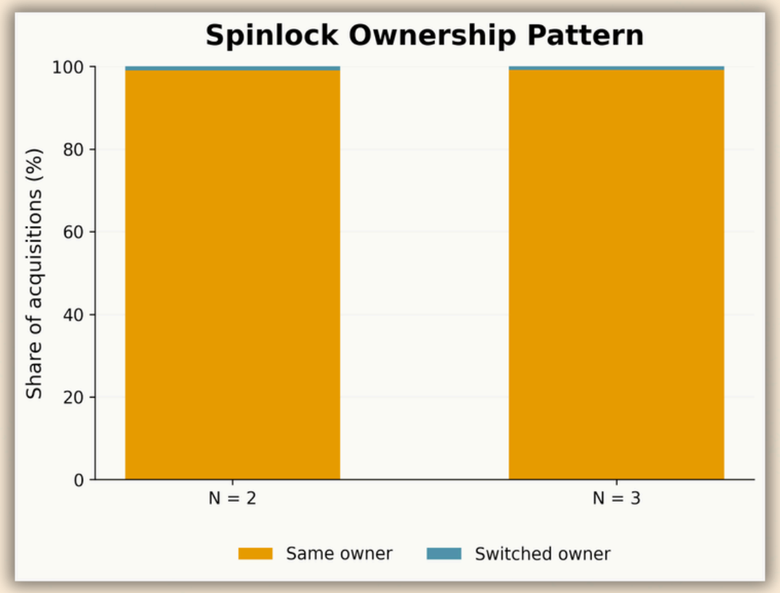
same-owner acquisitions

Forced handoff p50

N=2: ~90 ns | N=3: ~94 ns

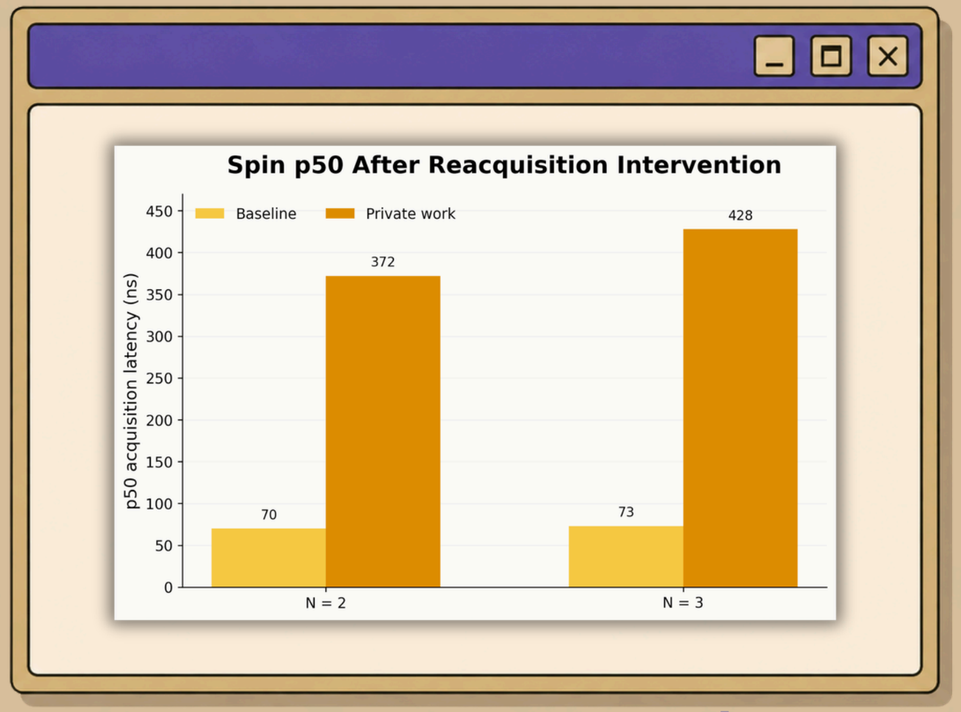
Natural switched-owner p50

N=2: 33.7 ***µs*** *| N=3: 38.7* ***µs***



REACQUISITION SENSITIVITY

What happens when immediate incumbent reacquisition is reduced?



Controlled Change

Private work added after unlock

Releasing thread is less able to immediately acquire lock

5.3×

at N=2 | 70 → 372 ns

5.9×

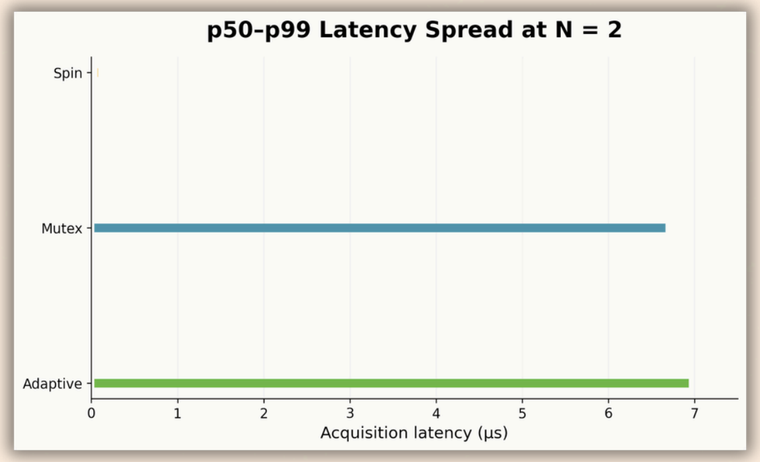
at N=3 | 73 → 428 ns

Reducing immediate reacquisition caused spin latency to rise ~5–6×, showing that incumbent reacquisition strongly affects the baseline result.

Analysis H1 → H2 → H3 → H4 → H5

H3 (D): SLEEP TAIL COST

Prediction: Mutex will have the largest p50–p99 spread.



~200× wider tail

blocking policies vs their p50

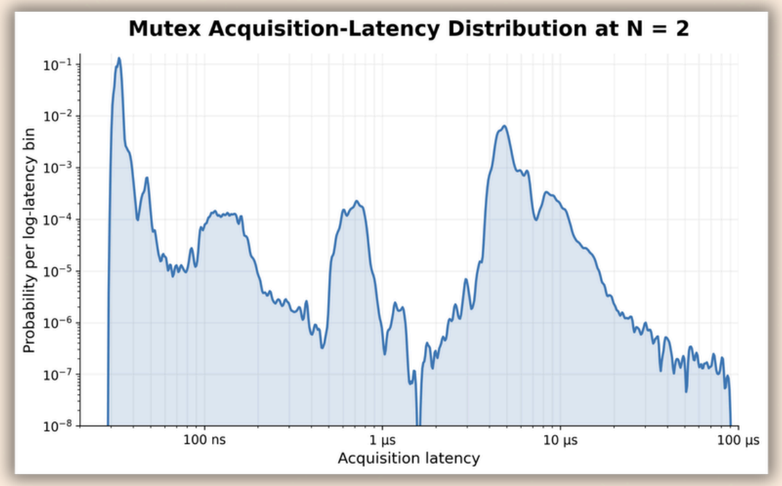
*Spin: 70 → 77 ns**
Mutex: 33 ns → 6.7 ***µs***
Adaptive: 34 ns → 7.0 ***µs***

*p50 → p99

Analysis | H1 → H2 → H3 → H4 → H5

H3 (II): MUTEX LATENCY DISTRIBUTION

Prediction: Mutex latency should split into fast and blocking populations.



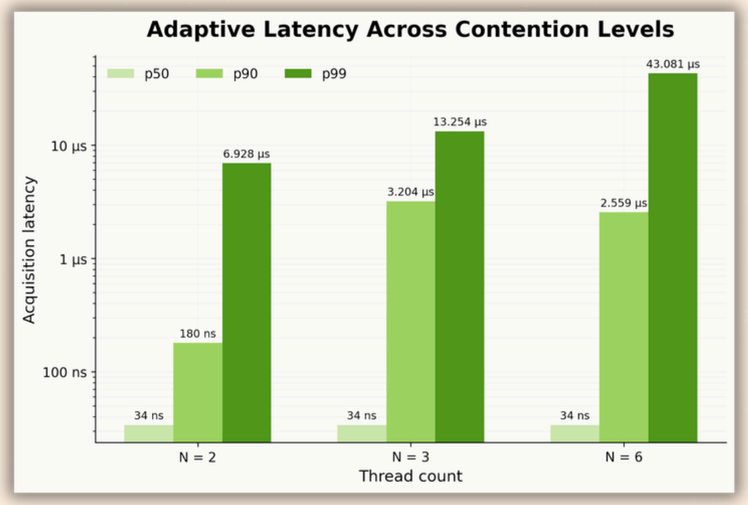
Fast mode: *~33 ns*
32-64 ns: *~80% of samples*

Slow mode: *~4.8 µs*
4-8 µs: *~11% of samples*

Analysis | H1 → H2 → H3 → H4 → H5

H4: ADAPTIVE INTERPOLATION

Prediction: Adaptive becomes more mutex-like as contention increases.



p50 ≈ 34 ns

Across $N = \{2,3,6\}$

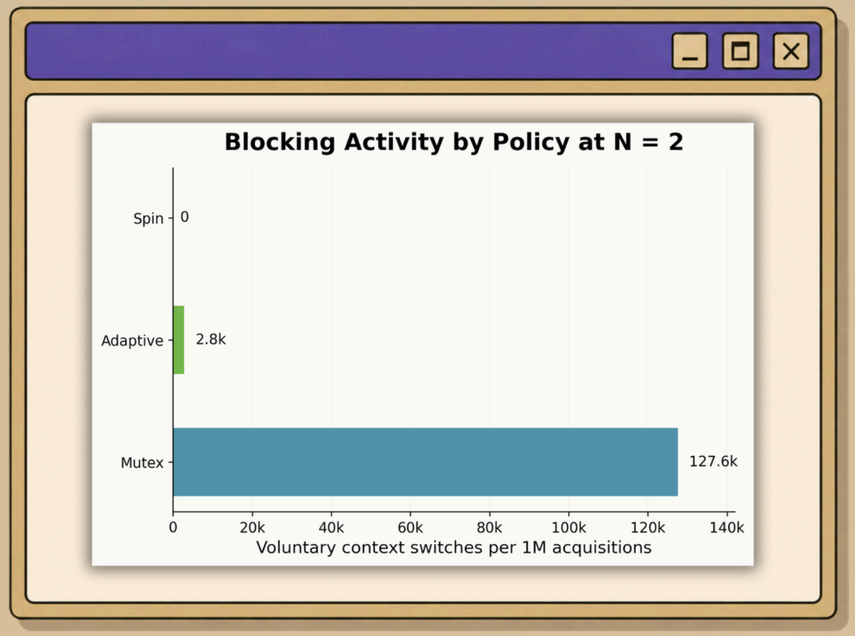
Upper-tail latency increases significantly with contention

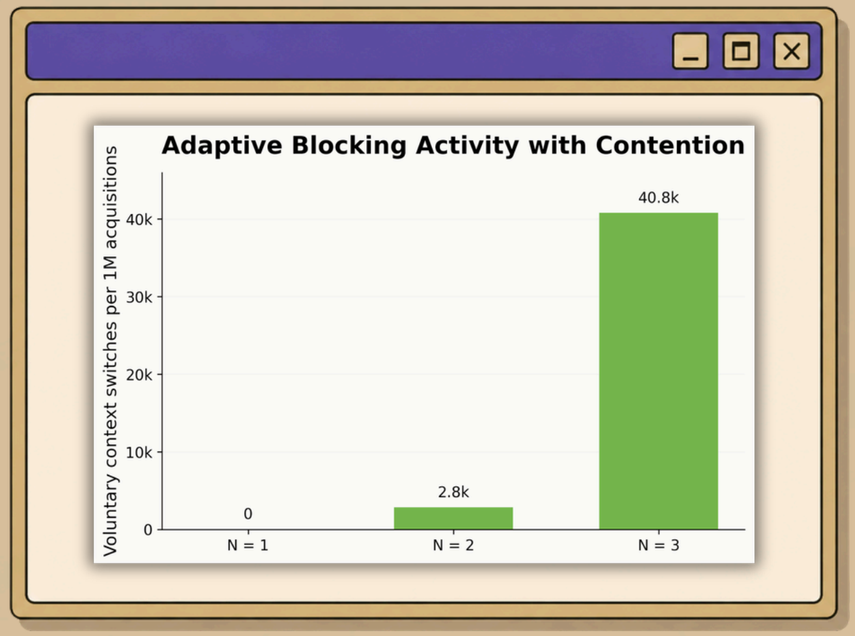
Analysis

H1 → H2 → H3 → H4 → H5

BLOCKING-PATH EVIDENCE

H3 + H4 Diagnosis



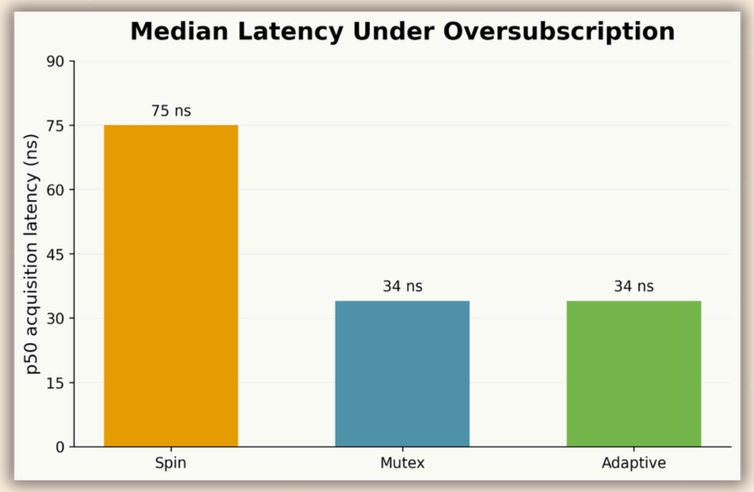


The latency tails align with blocking behaviour: Mutex blocks frequently, while Adaptive retains a fast path but increasingly falls back to sleeping.

Analysis | H1 → H2 → H3 → H4 → H5

H5: OVERSUBSCRIPTION REVERSAL

Prediction: Latency order reverses at N > C



No Reversal

Spin p50: *73 → 75 ns**

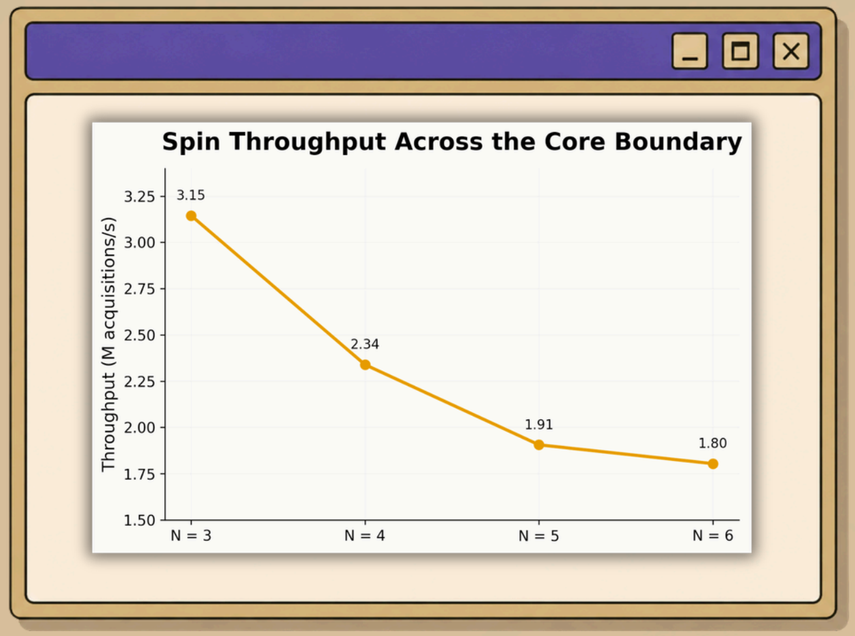
Spin p99: 83 → 83 ns

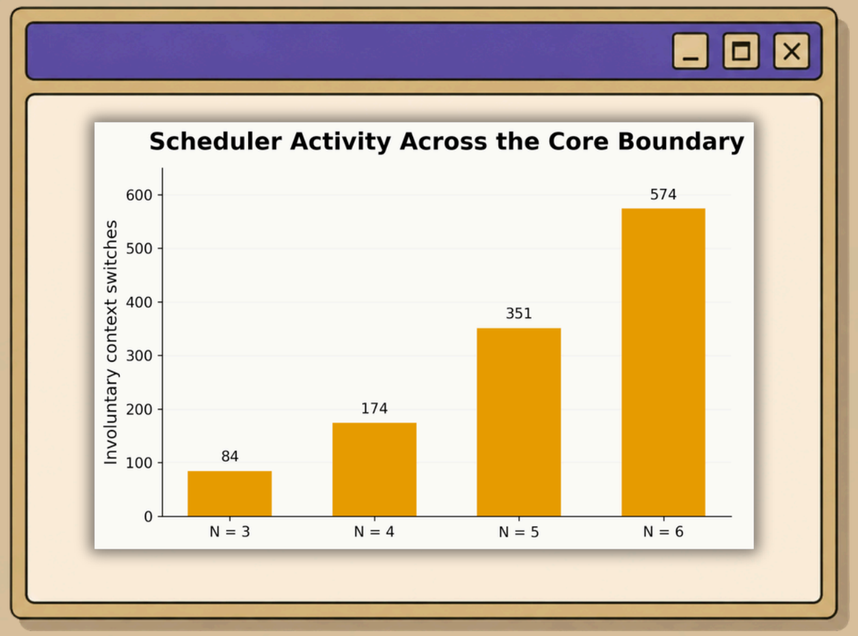
*N3 → N6

Analysis | H1 → H2 → H3 → H4 → H5

OVERSUBSCRIPTION BEYOND LATENCY

H5 Diagnosis





Oversubscription imposed a substantial cost, but it appeared as reduced throughput and increased scheduler activity rather than higher ordinary acquisition latency.

Analysis | H1 → H2 → H3 → H4 → H5

CONCLUSION

PROJECT CONCLUSION

05 - CONCLUSION

Objective

Evaluate spin, mutex, and adaptive waiting under increasing contention

Main Finding

No single wait policy was universally the best across thread counts

Unexpected Finding

Spin never gained the expected p50 advantage at N ≤ C due to reacquisition

Cost Distribution

Blocking increased tail latency and oversubscription reduced throughput

Question → Model → Hypothesis → Prediction → Test → Analysis → Conclusion

THANKS FOR LISTENING

Any Questions?